Resource contention in point to point overlapping GPU communication with MPI,NVSHMEM and NCCL

Aaron Haarti*
Department of Physics and Helsinki Institute of Physics, PL64, FI-00014 University of Helsinki, Finland
(Dated: Feb 2026)

Concepts: P2P, NCCL, NVSHMEM, GPU-Aware MPI, Overlapping computation and communication

I. INTRODUCTION

It is well known that to accurately model resource contention, it is necessary to first define how standard communication libraries interface with the hardware. The architectural paradigms of MPI, NCCL, and NVSHMEM dictate their reliance on the GPU's Streaming Multiprocessors (SMs) and High-Bandwidth Memory (HBM), which fundamentally drives the contention profile. We evaluate the performance impact of overlapping computation and communication by defining the execution models of these libraries.

In Sec. II we will introduce the mathematical resource contention analysis and the degradation factors. In Sec. III we will describe the methodology for simulating and measuring resource contention. In Sec. **??** we present measurement results. Finally, we conclude in Sec. **??**.

I.1. Communication Library Execution Models

We specifically investigate the resource contention profiles of point-to-point (P2P) communication primitives. While collective operations are highly relevant and pervasive in distributed deep learning, our study isolates P2P calls to precisely quantify hardware bottlenecks without the compounding complexity of reduction trees or all-to-all topologies.

I.1.1. GPU-Aware MPI

Modern implementations of the Message Passing Interface (MPI), such as MVAPICH2-GDR or OpenMPI, utilize Unified Virtual Addressing (UVA) and GPUDirect RDMA [1, 2]. In this host-initiated paradigm, the CPU orchestrates P2P operations like `MPI_Send` or `MPI_Recv`. The Network Interface Card (NIC) accesses the GPU's HBM directly via the PCIe switch, entirely bypassing the GPU's SMs. Because the SMs are not involved in data staging or transmission, this model inherently isolates compute resources from communication traffic [3].

I.1.2. NVIDIA NCCL

While primarily recognized for producing highly optimized collective communications (e.g., `ncclAllReduce`), the NVIDIA Collective Communication Library (NCCL) also provides a robust P2P API (`ncclSend` and `ncclRecv`), which is the main focus of our study. NCCL operates on a host-initiated, kernel-executed model [4]. According to NCCL architecture documentation, communication is managed by mapping thread blocks to SMs, referred to as "Channels." Both P2P and collective routines employ a copy-based pipeline: the SMs actively copy data from the user buffer into internal FIFO staging buffers in the HBM, perform any necessary mathematical reductions (for collectives), and then transmit the chunks to the NIC [5, 6]. This approach requires active SM cycles and effectively doubles the HBM bandwidth requirement for a given payload [7]. We note that empirically, NCCL typically consumes between 4 to 24 SMs depending on the number of initialized parallel channels, a footprint that can be manually constrained via the `NCCL_MAX_CTAS` environment variable [8]. Even if SMs are artificially constrained, the heavily memory-intensive nature of the internal staging kernels means that overlapping NCCL P2P transfers with dense compute kernels (e.g., GEMMs) frequently results in severe HBM bandwidth starvation. We observe that this contention can drop the Model Flops Utilization (MFU) of the compute kernel dramatically (e.g., from 80% down to 50%) entirely due to memory contention rather than SM starvation [9].

I.1.3. NVIDIA NVSHMEM

NVSHMEM implements a Partitioned Global Address Space (PGAS) designed for high-performance point-to-point memory access [10]. While it supports device-initiated communication invoked directly from within the CUDA compute kernel, our study focuses on its host-orchestrated, stream-enqueued P2P APIs (e.g., `nvshmemx_putmem_nbi_on_stream`). Unlike MPI or NCCL, the GPU threads directly calculate destination addresses and ring the network doorbell to trigger the NIC [6]. While this eliminates CPU orchestration overhead and delivers ultra-low latency for fine-grained messages, it requires the SMs to actively spend cycles issuing network requests rather than performing arithmetic computation [11]. The exact SM involvement strictly de-

* aaron.haarti@helsinki.fi

pends on the transport layer: over intra-node NVLink, SMs perform active load/store instructions (high SM involvement); over InfiniBand (IBGDA), SMs simply write Work Queue Elements (WQEs) to the NIC (lower SM footprint); and on newer architectures like Hopper, the Tensor Memory Accelerator (TMA) handles the transfer autonomously, dropping SM involvement to near-zero [12]. We compute that despite variable SM usage, stream-enqueued `put` operations actively read data from the local HBM to send it. NVLink IPC transfers, for instance, can reach ∼75–80% of the theoretical peak bandwidth, heavily saturating the shared HBM read ports. Consequently, overlapping NVSHMEM P2P operations with memory-bound compute kernels still results in significant hardware contention [10].

II. RESOURCE CONTENTION ANALYSIS

We evaluate the performance impact of overlapping computation and communication. We use multiplicative degradation factors to model resource contention realistically, as contention typically degrades the rate of execution, stretching task durations rather than adding a flat penalty.

Let E be the efficiency which measures the sequential execution time relative to the overlapped execution time. Let T_S be the total sequential time, T_K be the computation time, T_C be the communication time, and T_O be the total overlapping time:

$$E = \frac{T_K + T_C}{T_O}$$

$$T_K + T_C \equiv T_S \implies E = \frac{T_S}{T_O}.$$

We define the loss function due to resource contention, L, which accounts for the ratio between the theoretical perfectly overlapped time and the actual time including boundary computation T_B. Assuming boundary computation is zero ($T_B = 0$), we compute:

$$L = \frac{\max(T_K, T_C)}{T_O}.$$

Ideal: In a perfect system without contention, the degradation factors are 1. The loss function evaluates to 1, and the total time is strictly the maximum of computation or communication.

$$L = 1 \implies E = \frac{T_K + T_C}{\max(T_K, T_C)}.$$

Contention: When contention exists, it acts as a penalty factor that increases the total time of T_K and T_C individually. Let $a_{CL} \geq 1$ be the computation degradation factor and $b_{CL} \geq 1$ be the communication degradation factor. The total overlapping time T_O becomes the maximum of these stretched times:

$$T_O = \max(T_K \cdot a_{CL}, T_C \cdot b_{CL}).$$

The degradation factors a_{CL} and b_{CL} are influenced by resource contention between the concurrent communication and compute kernels, depending on the RDMA engine, bandwidth, arithmetic intensity, and message size.

II.1. Hardware-Dependent Formulation of Degradation Factors

We compute the degradation factors a_{CL} and b_{CL} to quantify the structural bottlenecking of the GPU's Streaming Multiprocessors (SM) and High-Bandwidth Memory (HBM). Let C_{SM} and C_{HBM} denote the total hardware capacities for compute and memory bandwidth, respectively.

II.1.1. Resource Demands

A compute kernel K with a specific arithmetic intensity A demands resources (R^K_{SM}, R^K_{HBM}).

A communication task C handling message size M demands resources (R^C_{SM}, R^C_{HBM}). The magnitudes of R^C_{SM} (software overhead and staging) and R^C_{HBM} (physical DMA memory access) depend strictly on the transport mechanism utilized (e.g., copy-based versus zero-copy RDMA).

II.1.2. Computation Degradation (a_{CL})

The computation degradation factor a_{CL} is driven by SM thread starvation and HBM bandwidth saturation. We model it as the maximum resource congestion ratio:

$$a_{CL} = \max\left(1, \frac{R^K_{SM} + R^C_{SM}}{C_{SM}}, \frac{R^K_{HBM} + R^C_{HBM}}{C_{HBM}}\right).$$

Impact of RDMA: When utilizing zero-copy RDMA (e.g., GPU-Aware MPI or NCCLX), the communication task bypasses the SMs, yielding $R^C_{SM} = 0$. If the compute kernel is compute-bound (high A, requiring high R^K_{SM} but low R^K_{HBM}), bypassing the SM prevents contention, ensuring $a_{CL} \approx 1$. Conversely, if the communication uses a copy-based protocol (e.g., standard NCCL), $R^C_{SM} > 0$, forcing $a_{CL} > 1$.

II.1.3. Communication Degradation (b_{CL})

The communication degradation factor b_{CL} represents the stretching of the network transfer time due to HBM contention throttling the NIC's injection rate. We evaluate it as:

$$b_{CL} = \max\left(1, \frac{R^K_{HBM} + R^C_{HBM}}{C_{HBM}}\right).$$

Impact of Arithmetic Intensity: If the overlapped compute kernel is memory-bound (low arithmetic intensity A), it demands massive HBM bandwidth ($R^K_{HBM} \rightarrow$

C_{HBM}). This throttles the NIC's ability to fetch data via DMA from the GPU, forcing $b_{CL} > 1$ regardless of the RDMA engine's theoretical capacity.

III. METHODOLOGY: SIMULATING RESOURCE CONTENTION

To empirically measure the resource degradation factors—specifically the Streaming Multiprocessor contention (a_{CL}) and High-Bandwidth Memory contention (b_{CL})—we develop a highly tunable benchmark kernel. Rather than executing a fixed application, this kernel acts as a synthetic stress test where the arithmetic intensity (A) can be dynamically modulated. This allows for the precise isolation of SM and HBM bottlenecks during overlapping communication.

III.1. Modulating Arithmetic Intensity (A)

To produce varying arithmetic intensity, we introduce the `math_per_load` parameter, which dictates the ratio of mathematical operations executed by the SMs relative to the memory fetches executed by the memory controller.

- **Memory-Bound Workloads:** A low `math_per_load` forces the kernel to fetch data from HBM on nearly every loop iteration. This maximizes the compute kernel's memory demand (R^K_{HBM}), stressing the memory bandwidth and allowing the measurement of b_{CL} when overlapped with DMA traffic.

- **Compute-Bound Workloads:** A high `math_per_load` executes hundreds of arithmetic instructions for every memory fetch. This maximizes SM demand (R^K_{SM}) while generating minimal memory traffic, allowing the measurement of a_{CL} when overlapped with copy-based communication protocols.

III.2. Simulating SM Contention

To ensure genuine SM utilization, we force the execution of a continuous chain of dependent Fused Multiply-Add (FMA) instructions in the kernel. Because the current value of the accumulator depends strictly on the immediate previous calculation, the compiler cannot unroll or optimize the loop away. This guarantees that the SMs are physically occupied for the specified number of iterations, creating a heavy compute footprint that will physically slow down if a communication operation attempts to steal SM cycles.

III.3. Simulating HBM Contention

To generate realistic memory pressure, we avoid simple contiguous memory reads, which the GPU's L1/L2 caches would easily absorb. Instead, we periodically read from neighboring lattice sites based on 4D grid strides. By jumping through memory using varying strides, we force cache misses and guarantee that data must be pulled directly from the physical HBM. If a concurrent RDMA transfer is also pulling from HBM, the memory controller will throttle both tasks, exposing the true hardware memory bottleneck.

III.4. Compute Kernel Implementation

In Listing 1 we detail the benchmark compute kernel. We structure the code to enforce dependent mathematical operations while conditionally triggering strided memory accesses based on the target arithmetic intensity.

[1] S. Potluri, K. Hamidouche, A. Venkatesh, D. Bureddy, and D. K. Panda, Efficient inter-node mpi communication using gpudirect rdma for infiniband clusters with nvidia gpus, in *2013 42nd International Conference on Parallel Processing* (IEEE, 2013) pp. 80–89.

[2] NVIDIA Corporation, Developing a Linux Kernel Module using RDMA for GPUDirect (2024), accessed: 2026-04-19.

[3] C.-H. Chu *et al.*, Exploiting hardware multicast and gpudirect rdma for efficient broadcast, in *IEEE Transactions on Parallel and Distributed Systems* (IEEE, 2020).

[4] NVIDIA Corporation, NVIDIA Collective Communication Library (NCCL) Documentation (2024), accessed: 2026-04-19.

[5] M. Si, P. Balaji, Y. Chen, C.-H. Chu, A. Gangidi, *et al.*, Collective communication for 100k+ gpus (2025), details the exact copy-based pipeline and FIFO staging of baseline NCCL., arXiv:2510.20171 [cs.DC].

[6] D. Unat, I. Turimbetov, M. K. T. Issa, D. Sağbili, F. Vella, D. De Sensi, and I. Ismayilov, The landscape of gpu-centric communication, arXiv preprint arXiv:2409.09874 (2024).

[7] A. Agrawal, S. Aga, S. Pati, and M. Islam, Optimizing ml concurrent computation and communication with gpu dma engines, in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)* (2025) quantifies HBM and SM contention caused by copy-based collectives versus DMA offloading.

[8] NVIDIA Corporation, NCCL Environment Variables (2024), accessed: 2026-04-19.

[9] S. Lee, J. Oh, J. Kim, S. Go, J. Park, and D. Mahajan, Characterizing compute-communication overlap in gpu-accelerated distributed deep learning: Performance and power implications (2025).

[10] NVIDIA Corporation, NVSHMEM Developer Guide (2024), accessed: 2026-04-19.

```cpp
template <typename T>
__global__ void kernel_compute(T *d_bulk, const size_t *grid_size, size_t volume,
                               int iters, int math_per_load) {
    size_t idx = blockIdx.x * (size_t)blockDim.x + threadIdx.x;
    if (idx >= volume) return;

    // Load grid dimensions into registers
    size_t g0 = grid_size[0], g1 = grid_size[1], g2 = grid_size[2];

    // Initial load
    T res = d_bulk[idx];
    T factor = (T)1.000001;

    // Strides to defeat simple contiguous caching
    size_t s0 = 1, s1 = g0, s2 = g0 * g1, s3 = g0 * g1 * g2;
    int load_count = 0;

    // HEAVY COMPUTE & MEMORY LOOP
    for (int i = 0; i < iters; ++i) {
        // 1. Dependent FMA ensures continuous SM utilization
        res = (res * factor) + (T)0.0123;

        if (res > 1e10 || res < -1e10) res *= 1e-10;

        // 2. Modulated HBM fetches based on Arithmetic Intensity
        int m_load = (math_per_load > 0) ? math_per_load : 1;
        if (i % m_load == 0) {
            size_t neighbor_idx;
            int mode = load_count % 4;

            if (mode == 0) neighbor_idx = (idx + s0) % volume;
            else if (mode == 1) neighbor_idx = (idx + s1) % volume;
            else if (mode == 2) neighbor_idx = (idx + s2) % volume;
            else neighbor_idx = (idx + s3) % volume;

            res += d_bulk[neighbor_idx];
            load_count++;
        }
    }
    // Final dependent write
    d_bulk[idx] = res;
}
```

Listing 1: Benchmark compute kernel simulating arithmetic intensity and HBM contention.

[11] C. Hwang, K. Park, R. Shu, X. Qu, P. Cheng, and Y. Xiong, ARK: GPU-driven code execution for distributed deep learning, in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)* (USENIX Association, 2023) pp. 87–101, analyzes SM cycle consumption during device-initiated communication.

[12] NVIDIA Corporation, NVIDIA Hopper Architecture In-Depth (2022), accessed: 2026-04-19.